Cloud-Agnostic, Automated, Zero-Downtime Deployment Platform

Architecture document (max 5 pages)

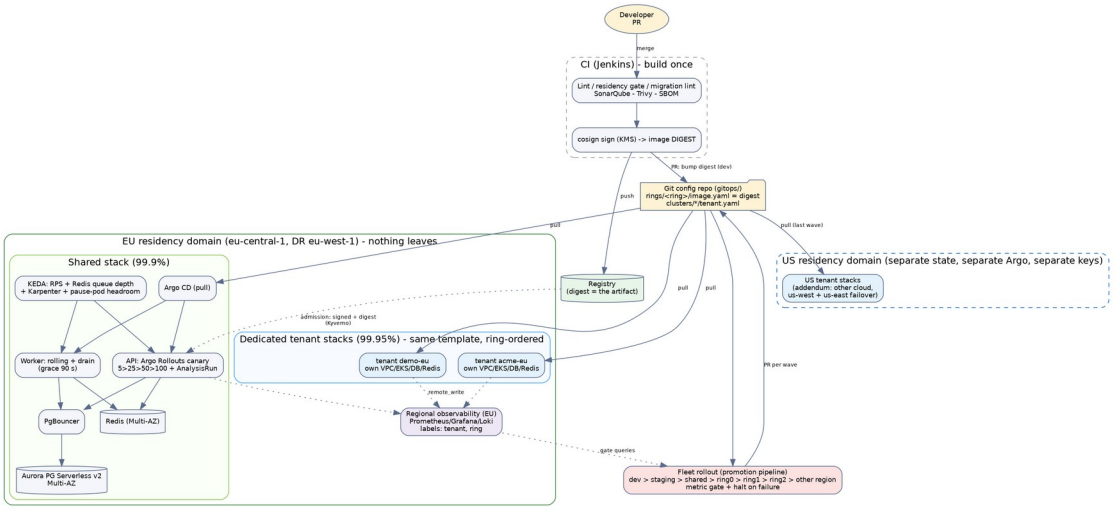
1. Scope and honesty statement

The sandbox used to author this had no Terraform, Kubernetes, Docker or network access. Everything is therefore labelled **RAN** (executed, output in `docs/evidence/out/`) or **NOT RUN** (written and reviewed, not executed). Full list: `docs/RUN-STATUS.md`.

Area	Status
Tenant schema + EU residency gate, tenant generator, migration lint, fleet rollout (halt-on-failure, dry-run), capacity and cost models, stand-in app	**RAN**: 16 unit tests pass; outputs saved
Terraform (AWS + kind provider, platform, tenant module, bootstrap), Helm chart, Jenkins, Argo/KEDA/Kyverno manifests, k6, restore test	**NOT RUN**: authored carefully, never applied or rendered. Run `make lint` first
Deploy-under-load, auto-rollback, 15x test, restore test recordings	**NOT RUN**: scripts ready (`scripts/demo-*.sh`, `restore-test.sh`); evidence to be recorded on a machine with Docker

The application is a black box. Assumptions about it are numbered A1 to A4 and drive the risks in section 7.

2. Architecture



Platform architecture

Layers. `infra/provider/*` (network, Kubernetes, Postgres, Redis, secrets) is the only cloud-specific code and obeys a fixed output contract (`provider/CONTRACT.md`). `infra/platform` (ingress, Argo CD and Rollouts, KEDA, Prometheus, Grafana, Loki, External Secrets, Kyverno) and `gitops/charts/app` (the application) consume only the contract

outputs. Two targets: **AWS** (EKS, Aurora, ElastiCache) and **kind** (same platform module, in-cluster Postgres and Redis). Non-portable items are listed in `docs/NOT-PORTABLE.md`.

Automation. `infra/bootstrap/aws` turns an empty account into remote state (versioned, KMS-encrypted S3 with native locking) and a GitHub-OIDC CI role. After that no console steps are needed: `scripts/tf-apply.sh <stack>` applies cluster first, then platform; Argo CD then pulls everything else. Secrets are generated by Terraform into Secrets Manager and reach pods through External Secrets (nothing in git). Drift is detected nightly by `Jenkinsfile.infra` (`plan -detailed-exitcode`, Slack alert on exit 2) and Argo `selfHeal` reverts in-cluster drift.

Pipeline. Jenkins builds **once**, scans (Trivy, SonarQube, IaC scan), writes an SBOM, signs the **digest** with a KMS-held cosign key and opens a PR bumping `gitops/rings/dev/image.yaml`. Promotion is only ever a PR that moves the same digest forward; Kyverno rejects unsigned or tag-referenced images at admission. Gates between waves query Prometheus (5xx ratio < 0.5%, p95 < 400 ms) and **fail closed** when there is no data.

3. Deployment strategy and zero downtime

API: canary via Argo Rollouts, traffic 5 > 25 > 50 > 100 %, 5 min pause per step. An `AnalysisTemplate` runs every 60 s on the canary pods only (selected by pod-template-hash): 5xx ratio <= 0.5%, p95 <= 0.4 s, zero restarts; one failure aborts and returns 100% to stable automatically. **Worker: rolling with draining** (`maxUnavailable: 0`, `terminationGracePeriodSeconds: 90` > 60 s max job): workers pull work, so there is no traffic to split; safety comes from draining and from the metrics gate on the API/queue. **GitOps pull** (Argo CD in each cluster): no inbound cluster credentials, audit trail in git, drift correction. **Order:** dev > staging > shared-prod > ring0 (internal) > ring1 (small) > ring2 (large) > other region; `fleet_rollout.py` halts on the first red wave and never touches later ones (demonstrated in `fleet-rollout-halt-dryrun.txt`).

Zero-downtime mechanics. `startupProbe` plus `/readyz` readiness; a 10 s `preStop` sleep so ingress deregisters before SIGTERM; API fails readiness on SIGTERM and drains (stand-in app proves the pattern); PDBs (`api` 10%, `worker` 1, `pgbouncer` min 2); 3-AZ topology spread; maxUnavailable 0.

Migrations (hardest item). Startup migrations, some non-backward-compatible, mean two versions can touch one schema. Controls: (1) CI `migration_lint.py` blocks destructive SQL unless the file is explicitly a reviewed `-- migration: contract` change; (2) policy: **expand > dual-write/backfill > contract** across at least two releases, so the previous version always works on the new schema and rollback is a plain digest revert; (3) migrations run once as an Argo PreSync Job on a direct DB connection under a Postgres advisory lock, while app pods get `RUN_MIGRATIONS=false`; (4) PITR is the last-resort rollback for a contract mistake. (2) and (3) depend on assumption A1 (risk R1).

4. Autoscaling and capacity (assumptions explicit; to be replaced by k6 measurements)

API: 40 req/s per 1 vCPU pod at ~70% CPU gives 4 pods at baseline (floor 6 for 3 AZs) and 57 pods at 2,250 req/s, **68 with 20% headroom**. Workers: 20,000 jobs/h = 5.6 jobs/s; with average job 20 s (assumed) that is 111 concurrent jobs, with the **60 s worst case 333** slots, or 84 pods at 4 slots each; max set to 90. DB: 68 x 10 + 90 x 2 = **~850 client connections** versus roughly 150 server connections through PgBouncer (transaction pooling); estimated 7 DB vCPU at peak, about 29 ACU, so Aurora max is 48.

Scale-up timeline to be validated (design, T = spike): KEDA polls every 15 s and HPA may add +100% or 30 pods per 15 s; **pause-pod headroom (~20 vCPU)** is preempted instantly, so about 20 pods start on warm nodes by T+35 s; Karpenter launches the remaining nodes in parallel (about 90 to 110 s), pods ready by about T+180 s, worker KEDA (Redis `listLength` 8 per pod) follows, steady by about **T+240 s**. Scale-down: API stabilisation 10 min and 10% per minute; workers 1 pod per minute, each drains for up to 90 s, so no job is dropped by autoscaling (spot interruptions get the same drain).

5. Multi-deployment and operations

One tenant is one reviewed PR: `scripts/new_tenant.py` writes `tenant.yaml` (name, residency, region, size, ring), the Terraform stack and the Argo Application together. CI validates schema and **residency** (EU tenant: region and DR must be EU; EU stack files may not mention US regions; tests included), `terraform plan` is policy-checked (`residency.rego`), and an AWS SCP (`eu-residency-scp.json`) denies out-of-EU regions and cross-border replication at the account level. Each residency domain has its own state bucket, KMS keys and Argo CD, so US operations never need EU credentials. Estimated provisioning, **not measured**: VPC 3 min, EKS 10 min plus nodes 3 min, Aurora in parallel 12 to 15 min, platform Helm 6 min, first sync 3 min, so about 25 to 30 minutes critical path (risk R5).

SLOs: shared 99.9% (43 min/month budget), dedicated 99.95% (22 min); multi-window burn-rate alerts (14.4x fast / 6x slow) per `tenant` label, p95 > 400 ms alert, queue-growth alert, backup-freshness alert, rollout-degraded alert. Per-tenant dashboard and `tenant` and `ring` labels on every series (dedicated clusters remote-write to the in-region Prometheus). Runbooks: bad release, migration failure, queue backlog, restore, tenant onboarding, region failover. Backups: Aurora automated backups with 14-day PITR, daily logical dump into a same-residency bucket, `restore-test.sh` (restore into a fresh instance, compare row counts and checksums).

6. Decisions and rejected alternatives

#	Decision	Rejected alternatives and why
D1	**Argo CD per cluster (pull)**; promotion = PR on a digest	*Jenkins push:* needs long-lived cluster credentials, no drift correction. *Central hub Argo:* single credential store for all tenants and cross-residency control plane
D2	**API canary (Argo Rollouts); worker rolling with drain**	*Blue/green:* doubles API capacity at 15x (cost) and still shares one DB, so two versions hit one schema anyway. *Plain rolling for API:* no metric gate, no automatic abort. *Canary for workers:* no traffic to split

#	Decision	Rejected alternatives and why
D3	**Aurora PostgreSQL Serverless v2** (writer + reader)	*Fixed RDS sized for peak:* pays for ~29 ACU-equivalent 24/7 against ~4 needed at baseline. *Scheduled or manual resize:* minutes of disruption, cannot follow a 5-minute spike. *Self-managed Postgres on K8s:* DR and backups become our burden
D4	**KEDA + Karpenter + pause-pod headroom**	*HPA + Cluster Autoscaler:* node-group scaling is typically slower than Karpenter's direct EC2 launch and has no queue-depth scaler built in. *Over-provision to peak:* peak needs ~8.6x the baseline vCPU (116 vs 13.5, `capacity.py`), paid all month
D5	**Dedicated tenant = own VPC, cluster, DB, Redis** from one module	*Namespace per tenant:* noisy neighbour, shared blast radius, cannot honestly promise 99.95%. *Account per tenant:* best isolation but account-vending overhead; kept as the upgrade path for regulated tenants
D6	**Terraform in three layers**, Helm for the app	*Crossplane:* larger operational surface for a small team. *CloudFormation/CDK:* AWS-only. *Single flat stack:* provider code leaks into platform and app
D7	**Expand/contract + pre-sync migration Job with advisory lock**	*Migrate on every pod start:* races and mid-rollout schema changes. *Blue/green databases:* data divergence

7. Top 5 risks

#	Risk	Mitigation (and what is still open)
R1	**A1: the black-box app may have no migrate-only mode** and its startup migrations cannot be disabled; rollback after a breaking migration is unsafe	Lint gate, expand/contract policy, PITR, canary starts at 5% so a bad migration is seen early. If the image cannot be told to skip migrations, the first canary pod migrates; this must be tested in staging before any production use
R2	**A2: worker semantics unknown** (SIGTERM drain, at-least-once, idempotency); Redis list queue can lose in-flight jobs on failover	90 s grace; drain test in the evidence plan; ask app team for idempotent jobs and ack-based reliable queue; MemoryDB considered but rejected on cost
R3	**Peak DB behaviour is unmeasured**: ACU estimate rests on an assumed 3 ms DB CPU per request; PgBouncer transaction mode may break session state or prepared statements (A4)	The 15x k6 test is the validation; max ACU 48 leaves headroom; fallback is session pooling with a larger pool
R4	**5-minute scale-up depends on EC2 capacity**, spot interruptions and image pull time	Multiple instance families, on-demand for API, headroom pods, pre-pulled AMI; quota increases requested in the 90-day plan
R5	**30-minute tenant SLA and operational maturity**: Aurora and EKS creation times vary; Terraform needs a two-phase apply (cluster, then Helm provider); the bootstrap CI role is Administrator for the demo	Parallelised resources, `tf-apply.sh`, timing measured at the first real tenant; scope the CI role per stack (roadmap days 1 to 30)

Top-five budget view: shared steady about USD 3.9k (limit 9k), shared month with 60 h at 15x about USD 5.1k (limit 16k), dedicated tenant size m about USD 1.5k each, room for three at steady state (`docs/COST.md`; unit prices approximate and unverified).